S.E. Assignment # 03
Whitebox Testing Report

Muhammad Mubashir Islam – 23i-0834
Muhammad Qasim – 23i-0552
Zia-ul-Baqa – 23i-0606
CS-D

Faculty Feedback and Evaluation
This document provides the detailed Whitebox testing results for a **cumulative <u>three</u> functions** from the controllers. ***<u>They use no external tools.</u>*** For this, we've narrowed down the **administrative module**, for the reasons that it **contains relatively large number of predicates**, and have calculated the cyclomatic complexity, control flow graphs, and the basis of independent sets. Furthermore the Junit tests are generated for testing the input validation methods.

We've used functions from **<u>controllers</u>** since they contain relatively large margin for errors, and also have a **lot of predicates**, which increases their cyclomatic complexity.

For the CFGs below, we've taken them so that each IF, WHILE becomes **ONE** predicate node, not compounded based on the number of conditionals in it. The return nodes are labelled 'R' and the normal statements are 'N.'

Function # 01: validateUser()

This function takes the credentials from registering a user as input, and performs business checks on it, on

```
private String validateUser(String accountName, String username, String loginID,
                            String role, String department){
    if(accountName.isEmpty() || username.isEmpty() || loginID.isEmpty() || role == null || role.isEmpty()){
        return "All fields must be filled.";
    }
    if(("HOD".equals(role) || "TEACHER".equals(role)) &&
            (department == null || department.isBlank())) {
        return "Department is required for HOD and Teacher accounts.";
    }
    if(role.equalsIgnoreCase("Student")){
        if(!username.matches("^[a-zA-Z]\\d{6}$")){
            return "Student ID must start with the campus initials, followed by batch-no, and roll-no (i.e. 23I-0606)";
        }
    }
    else if(role.equalsIgnoreCase("Teacher")){
        if(!username.matches("^[a-zA-Z]+\\.[a-zA-Z]+$")){
            return "Teacher ID must be in the format: [FirstName] . [LastName]";
        }
    }
    return null;
}
```

the Student or Faculty ID format.

<u>Control Flow Graph:</u>
* For this, we have taken the entire conditional inside the IF as a single predicate node.

- **D1**: if(accountName.isEmpty() || ... || role.isEmpty())
- **R1**: return "All fields must be filled."
- **D2**: if(("HOD".equals(role) || ...) && (department == null || …))
- **R2**: return "Department is required…"

- **D3**: if(role.equalsIgnoreCase(“Student"))
- **D4**: if(!username.matches("...")) (Student ID check)
- **R3**: return "Student ID must start with..."
- **D5**: else if(role.equalsIgnoreCase(“Teacher"))
- **D6**: if(!username.matches("...")) (Teacher ID check)
- **R4**: return "Teacher ID must be in the format..."
- **R5**: return null
- **Exit**: End of method

‘D’ is a decision (predicate) node, ‘R’ is a return node.

Cyclomatic Complexity

* This shows the number of independent paths through the function.
* Number of decision nodes (P): 6 (D1, D2, D3, D4, D5, D6)
  Calculation: **V(G)=P +1=6+1=7**

Note: If each individual condition within compound predicates is evaluated independently (short-circuit logic), the complexity would be much much higher (around 14). For basic analysis, statement-level decision nodes are used.

Basis Set of Paths

Based on **V(G) = 7**, there are 7 independent paths in the basis set:

1. D1 → R1 → Exit (Missing fields)
2. D1 → D2 → R2 → Exit (Missing department for HOD/Teacher)
3. D1→D2→D3→D4→R3→Exit(Invalid StudentID)
4. D1→D2→D3→D4→R5→Exit(Valid Student)
5. D1→D2→D3→D5→D6→R4→Exit(Invalid teacherID)
6. D1→D2→D3→D5→D6→R5→Exit(Valid teacher)
7. D1 → D2 → D3 → D5 → R5 → Exit

<u>Function # 02:</u> saveQuestion()

```java
@FXML
private void saveQuestion() {
    String text   = questionTextArea.getText();
    String type   = questionTypeBox.getValue();
    String factor = questionFactorBox.getValue();

    if (text == null || text.isBlank() || type == null || factor == null) {
        setStatus(addQuestionStatusLabel, "* Please fill in all required fields.", true);
        return;
    }

    String metadata = buildMetadata(type);
    if (metadata == null) {
        setStatus(addQuestionStatusLabel, "* Please complete the type-specific fields.", true);
        return;
    }

    if (QuestionDAO.saveQuestion(text.trim(), type, factor, metadata)) {
        SceneTools.showAlert(Alert.AlertType.INFORMATION,
                "Success",
                "Question Saved",
                text + " has been saved with ");
        clearQuestionForm();
        refreshQuestionsTable();
        refreshAvailableQuestionsTable();
    } else {
        SceneTools.showAlert(Alert.AlertType.ERROR,
                "Error", "Can not save question.",
                "Check question text, type, performance factor or other input fields.");
    }
}
```

This function takes the question sequence from the UI, performs basic business checks on it, and saves it to the database.

<u>Control Flow Graph:</u>

- **N1**: Retrieve input strings text, type, factor
- **D1**: if (text == null || text.isBlank() || type == null || factor == null)
- **N2**: setStatus(...) and return (Missing fields)
- **N3**: String metadata = buildMetadata(type);
- **D2**: if (metadata == null)
- **N4**: setStatus(...) and return (Missing metadata)
- **D3**: if (QuestionDAO.saveQuestion(…))
- **N5**: Success Alert, clear form, and refresh tables
- **N6**: Error Alert
- **Exit**: End of method

<u>Cyclomatic Complexity:</u>

- Number of decision nodes (P): **3 (D1, D2, D3)**
- Calculation: **V(G) = P + 1 = 3 + 1 = <u>4</u>**

<u>Basis Set of Paths</u>

1. N1 → D1 → N2 → Exit (Missing required fields)

2. N1 → D1 → N3 → D2 → N4 → Exit (Missing type-specific fields)

3. N1→D1→N3→D2→D3→N5→Exit (Successfully saved)

4. N1→D1→N3→D2→D3→N6→Exit (DB Errore)

<u>Function # 03:</u> saveForm()

```java
@FXML
private void saveForm() {
    String name = formNameField.getText();
    String desc = formDescArea.getText();

    if (name == null || name.isBlank()) {
        setStatus(addFormStatusLabel, "* Form name is required.", true);
        return;
    }
    if (selectedQuestionIds.isEmpty()) {
        setStatus(addFormStatusLabel, "* Please add at least one question.", true);
        return;
    }

    if (QuestionnaireDAO.saveQuestionnaire(name.trim(), desc, new ArrayList<>(selectedQuestionIds))) {
        SceneTools.showAlert(Alert.AlertType.INFORMATION,
                "Success",
                "Questionnaire Saved",
                name + " has been saved with " + selectedQuestionIds.size() + " questions.");
        clearFormDraft();
        refreshFormsTable();
        refreshPushFormComboBox();
    } else {
        SceneTools.showAlert(Alert.AlertType.ERROR,
                "Error", "Can not save questionnaire.",
                "Save failed. Check question name, description, or select at least one question.");
    }
}
```

This function takes all questions from the fields, and performs basic checks before saving it in the database.

- **N1**: Retrieve name, desc
- **D1**: if (name == null || name.isBlank())
- **N2**: setStatus(...) and return (Missing name)
- **D2**: if (selectedQuestionIds.isEmpty())
- **N3**: setStatus(...) and return (No questions added)
- **D3**: if (QuestionnaireDAO.saveQuestionnaire(...))
- **N4**: Success Alert, clear form draft, and refresh
- **N5**: Error Alert (Save failed)
- **Exit**: End of method

<u>Cyclomatic Complexity:</u>
- Number of decision nodes (P): **3 (D1, D2, D3)**
- Calculation: **V(G) = P + 1 = 3 + 1 = <u>4</u>**

<u>Basis Set of Paths:</u>
1. N1 → D1 → N2 → Exit (Missing form name)
2. N1→D1→D2→N3→Exit (No questions added to the form)
3. N1 → D1 → D2 → D3 → N4 → Exit (Successfully saved form)
4. N1→D1→D2→D3→N5→Exit (Database save error)

Whitebox testing using Junit tests:

Function # 01: conatainsAlphanumeric(String s)

```
static boolean containsAlphanumeric(String s) {  13 usages
    if (s == null) return false;
    for (char c : s.toCharArray()) {
        if (Character.isLetterOrDigit(c)) return true;
    }
    return false;
}
```

This functions is called upon by various methods across different forms to test that if the input data contains alphanumeric characters or just gibberish characters.

Test Cases Tried:

1. Null → false (null-guard branch)
2. Empty string → false
3. Symbols only → false
4. Whitespace only → false
5. Single letter → true
6. Single digit → true
7. Mixed symbols and letter → true (first alphanumeric found)
8. Full alphanumeric string → true
9. Unicode letter → true

Function # 02: validateUser(…)

```java
static String validateUser(String accountName, String username,  13 usages
                           String loginID, String role, String department) {
    if (accountName.isEmpty() || username.isEmpty() || loginID.isEmpty()
            || role == null || role.isEmpty()) {
        return "All fields must be filled.";
    }
    if (!containsAlphanumeric(accountName)) {
        return "Account name must contain at least one letter or digit.";
    }
    if (("HOD".equals(role) || "TEACHER".equals(role) || "STUDENT".equals(role))
            && (department == null || department.isBlank())) {
        return "Department is required for Student, HOD, and Teacher accounts.";
    }
    if (role.equalsIgnoreCase( anotherString: "Student")) {
        if (!username.matches( regex: "^[a-zA-Z]\\d{6}$")) {
            return "Student ID must start with the campus initials, followed by batch-no, and roll-no (i.e. 23I-0606)";
        }
    } else if (role.equalsIgnoreCase( anotherString: "Teacher") || role.equalsIgnoreCase( anotherString: "HOD")) {
        if (!username.matches( regex: "^[a-zA-Z]+\\.[a-zA-Z]+$")) {
            return "User ID must be in the format: [FirstName] . [LastName]";
        }
    }
    return null;
}
```

This function takes the credentials from registering a user as input, and performs business checks on it, on the Student or Faculty ID format.

Test Cases Tried:

1. Valid lower-case initial + 6 digits
2. Valid upper-case initial + 6 digits
3. Too few digits (5) → invalid
4. Too many digits (7) → invalid
5. Starts with digit → invalid
6. Contains letters in digit part → invalid
7. Empty string → invalid
8. Valid format: firstname.lastname
9. Upper-case parts → valid
10. Missing dot → invalid
11. Trailing dot, no last name → invalid
12. Leading dot, no first name → invalid
13. Digit in first name → invalid
14. Multiple dots → invalid

15. Empty string → invalid

Function # 03: validateUser(…)

```
static int semesterYear(String semester) {  8 usages
    if (semester == null) return -1;
    String[] parts = semester.split( regex: " ",  limit: 2);
    if (parts.length < 2) return -1;
    try {
        return Integer.parseInt(parts[1].trim());
    } catch (NumberFormatException e) {
        return -1;
    }
}
```

This function takes input the semester string and extracts the year from it for form scheduling duration processing.

Test Case Tried:

1. null → -1 (null guard)
2. No space (single token) → -1
3. Normal 'Spring 2025' → 2025
4. 'Fall 2024' → 2024
5. 'Summer 2026' → 2026
6. Non-numeric year part → -1 (NumberFormatException caught)
7. Extra spaces in year part are trimmed → correct year

Other Tested Constraints:

1. Course field validation (AdminCourses)
2. Form schedule date-year constraint
3. Session – static state management
4. buildLikertMetadata(int, String)
5. buildSliderMetadata(String, int, int)
6. buildTextMetadata(String)

7. buildBinaryMetadata(String, String)
8. parseMetadata(String)